JavaScript Interview Questions & Answers

Part B — 40 important interview questions | Beginner-friendly answers with examples

Tip: Practise explaining each answer aloud in your own words. The examples are short so you can remember the concept and demonstrate it in an interview.

1. What are the different data types in JavaScript?

JavaScript has seven primitive types: string, number, bigint, boolean, undefined, symbol, and null. Objects are non-primitive values; arrays and functions are also objects (functions are callable objects). Example: `let name = "Sam";` creates a string, and `let items = [1, 2, 3];` creates an array.

2. Explain the difference between == and ===.

== compares values after possible type conversion, while === compares both value and type without type conversion. Example: `5 == "5"` is true, but `5 === "5"` is false. Prefer === in most cases for predictable comparisons.

3. What is NaN and how do you check for it?

`NaN` means "Not-a-Number." It is a special numeric value that can result from an invalid mathematical conversion, such as `Number("hello")`. Use `Number.isNaN(value)` to check specifically. Example: `Number.isNaN(Number("hello"))` returns true.

4. Explain the differences between var, let, and const.

`var` is function-scoped and can be redeclared; it is hoisted and initialized as undefined. `let` and `const` are block-scoped and cannot be redeclared in the same scope. A `let` variable can be reassigned; a `const` binding cannot. A const object's properties can still be changed.

5. What is the difference between null and undefined?

`undefined` usually means a value has not been assigned or is missing. `null` is an intentional empty value. Example: `let a;` gives undefined, while `let b = null;` explicitly sets b to no value.

6. What is type coercion? Give examples.

Type coercion is JavaScript's conversion of a value from one type to another. It can be implicit or explicit. Implicit example: `"5" + 2` produces `"52"`. Explicit example: `Number("5") + 2` produces 7.

7. Explain truthy and falsy values in JavaScript.

Truthy values behave like true in a condition; falsy values behave like false. Common falsy values are false, 0, -0, 0n, an empty string (""), null, undefined, and NaN. Most other values—including empty arrays `[]` and empty objects `{}`—are truthy.

8. What is a closure? Give a practical example.

A closure happens when a function remembers variables from the scope where it was created, even after that outer function has finished. Example: `function makeCounter(){ let count = 0; return () => ++count; }`. Calling `const counter = makeCounter(); counter();` returns 1, and the next call returns 2. Closures are useful for private state and callbacks.

9. What is the difference between a normal function and an arrow function?

Arrow functions have shorter syntax and do not create their own `this` or `arguments`; they use `this` from the surrounding scope. Regular functions have their own `this` depending on how they are called and can be used as constructors with `new`. Arrow functions cannot be constructors.

10. What is currying? Provide an example.

Currying transforms a function that takes multiple arguments into a sequence of functions, each taking one argument. Example: `const add = a => b => a + b;`. Then `add(2)(3)` returns 5. It is useful for creating reusable, partially configured functions.

11. What is an IIFE and why is it used?

IIFE means Immediately Invoked Function Expression. It is a function expression that runs as soon as it is defined. Example: `(function(){ const message = "Hello"; console.log(message); })();`. Historically, IIFEs helped create a private scope and avoid polluting the global scope; modules now provide another common way to do this.

12. Explain the concept of this in different contexts.

In a regular object method, `this` usually refers to the object before the dot, such as `user.sayHi()`. In a regular function, its value depends on how the function is called and the mode in use. In an arrow function, `this` is inherited from the surrounding scope rather than being set by the call.

13. What is the difference between call(), apply(), and bind()?

All three let you specify a function's `this` value. `call(thisArg, a, b)` invokes the function with arguments separately. `apply(thisArg, [a, b])` invokes it with arguments in an array-like list. `bind(thisArg)` returns a new function with the chosen `this` value and optionally preset arguments; it does not call the function immediately.

14. What is memoization? How can it be implemented?

Memoization stores the result of a function call so that the same input can return a cached result instead of repeating expensive work. Example: `const memoSquare = (() => { const cache = new Map(); return n => { if (cache.has(n)) return cache.get(n); const result = n * n; cache.set(n, result); return result; }; })();`. It is most useful for repeatable calculations with the same inputs.

15. What is a pure function?

A pure function returns the same output for the same inputs and has no observable side effects. Example: `function add(a, b) { return a + b; }`. A function that changes a global variable, modifies its input, or depends on the current time is not pure.

16. What is prototypal inheritance in JavaScript?

Prototypal inheritance allows an object to access properties and methods from another object through its prototype. This supports shared behavior without copying every method into every object. JavaScript's class syntax is built on top of this prototype system.

17. Explain the prototype chain.

When JavaScript cannot find a property directly on an object, it looks at that object's prototype, then at the prototype's prototype, and so on. This sequence is called the prototype chain. The search stops when the property is found or the chain reaches null.

18. Differentiate Object.create() and constructor functions/classes.

`Object.create(proto)` creates a new object with the specified prototype. A constructor function is called with `new` to initialize instances, and an ES6 class provides class-style syntax for creating instances and defining methods. Classes and constructor functions generally use prototypes for shared instance methods.

19. What is the difference between shallow copy and deep copy?

A shallow copy duplicates only the top-level object; nested objects are still shared references. A deep copy also duplicates nested data. Example: `const copy = {...original}` is shallow. `structuredClone(original)` can deep-copy many supported values, but not every possible value; check its limitations for special objects and functions.

20. How do ES6 classes relate to prototypes?

ES6 classes provide clearer syntax for JavaScript's prototype-based inheritance. Methods declared in a class are placed on the class's prototype and shared by its instances. Classes are not a completely separate inheritance system from prototypes.

21. Explain the JavaScript event loop.

JavaScript runs synchronous code on the call stack. Asynchronous operations are handled by browser or runtime APIs. When callbacks become ready, they wait in queues. The event loop coordinates when queued work can run after the current stack is clear. Promise reactions are processed as microtasks, generally before the next task such as a timer callback.

22. What is the difference between microtasks and macrotasks?

Microtasks include Promise `.then()`/`catch()`/`finally()` reactions and `queueMicrotask()`. Tasks (often called macrotasks) include callbacks from `setTimeout`, user events, and many I/O operations. After the current synchronous work, the runtime drains the microtask queue before moving to another task. Exact scheduling details can vary by environment.

23. What are Promises? Explain their states.

A Promise represents the eventual success or failure of an asynchronous operation. Its states are pending, fulfilled, and rejected. Use `then()` for fulfillment, `catch()` for errors, and `finally()` for cleanup that should run either way.

24. Explain async/await and how it relates to Promises.

`async` makes a function return a Promise. `await` pauses that async function until a Promise settles, then gives its fulfilled value or throws its rejection as an error. It makes Promise-based code easier to read. Use `try...catch` to handle errors.

25. What is callback hell and how can it be avoided?

Callback hell is deeply nested callbacks that make code difficult to read and maintain. It can be reduced by splitting work into named functions, using Promises and chaining `then()`, or using `async/await` with proper error handling.

26. What is the difference between setTimeout and setInterval?

`setTimeout(callback, delay)` schedules a callback to run once after at least the specified delay. `setInterval(callback, delay)` schedules repeated callbacks at roughly that interval until stopped with `clearInterval()`. Delays are not exact guarantees; busy execution can make callbacks run later.

27. What are debouncing and throttling? Where are they used?

Debouncing runs a function after events stop occurring for a chosen delay—for example, waiting until a user pauses typing before searching. Throttling limits a function to at most a chosen frequency—for example, handling scroll or resize events without running it on every event. Both can improve performance.

28. What is destructuring? Give examples.

Destructuring extracts values from arrays or properties from objects into variables. Array example: `const [first, second] = [10, 20];`. Object example: `const {name, age} = user;`. It can make code shorter and clearer.

29. What is the spread/rest operator?

Both use `...`, but their roles differ. Spread expands an iterable or object's properties, for example `const combined = [...a, ...b];`. Rest collects remaining values into an array or object, for example `function sum(...numbers) { }`. Context determines whether it is spread or rest.

30. Explain template literals.

Template literals use backticks and allow variables or expressions inside `${...}`, as well as multi-line strings. Example: `const message = `Hello, ${name}!`;`. They are useful for building readable strings.

31. What are JavaScript modules (import/export)?

Modules let code be split into separate files with controlled exports and imports. Use `export` to make values available and `import` to use them elsewhere. Example: `export const add = (a, b) => a + b;`, then `import { add } from './math.js';`. In browsers, use `<script type="module">`.

32. What are optional chaining (?.) and nullish coalescing (??)?

Optional chaining safely accesses a property or method when the value before it may be null or undefined: `user?.address?.city`. Nullish coalescing provides a fallback only when the left side is null or undefined: `const name = inputName ?? "Guest";`. Unlike `||`, it does not replace valid values such as 0 or an empty string.

33. What is hoisting? Explain with var and let/const.

Hoisting describes how declarations are processed before code runs. A `var` declaration is available as undefined before its assignment. `let` and `const` are also registered before execution, but cannot be accessed before their declaration line because they are in the Temporal Dead Zone. Function declarations can generally be called before their definition.

34. What is the Temporal Dead Zone?

The Temporal Dead Zone (TDZ) is the period from entering a scope until a `let` or `const` declaration is initialized. Accessing that variable during the TDZ throws a ReferenceError. Example: `console.log(x); let x = 5;` throws an error.

35. Explain event bubbling and event delegation.

Event bubbling means an event typically travels from the target element up through its ancestors. Event delegation uses this behavior by adding one listener to a parent and checking which child triggered the event, often using `event.target` or `closest()`. It is useful for lists with many or dynamically added items.

36. What is the difference between synchronous and asynchronous code?

Synchronous code runs step by step, and each operation must finish before the next begins. Asynchronous code allows waiting work—such as fetching data or timers—to complete without blocking the rest of the program. JavaScript commonly handles this using callbacks, Promises, and `async/await`.

37. How does garbage collection work in JavaScript?

JavaScript automatically reclaims memory for objects that are no longer reachable by the program. Modern engines commonly use reachability-based methods, including tracing algorithms. Developers usually do not free memory manually, but should avoid keeping unnecessary references, such as forgotten event listeners or long-lived caches.

38. What are higher-order functions? Give examples.

A higher-order function takes another function as an argument, returns a function, or both. Array methods such as `map()`, `filter()`, and `reduce()` are common examples. `[1, 2, 3].map(n => n * 2)` returns `[2, 4, 6]`.

39. What is the difference between local storage, session storage, and cookies?

Local storage keeps string key/value data across browser sessions until cleared. Session storage normally lasts for the lifetime of a browser tab's page session. Cookies are small values that can be configured with expiry and security attributes and are often sent with matching HTTP requests. Do not store sensitive data carelessly in browser storage; cookie security settings and application design matter.

40. Explain deep equality and reference equality for objects.

Reference equality checks whether two variables refer to the exact same object. For example, `({a: 1}) === ({a: 1})` is false because these are separate objects. Deep equality checks whether their contents are equivalent according to a comparison rule. JavaScript has no single built-in deep-equality operator; the correct approach depends on the data types and requirements.

These answers follow the 40-question interview section in your uploaded question bank. Some concepts have environment-specific details, so keep examples simple and explain the main idea clearly.